SKILLARGER

Learn. Build. Get Hired.

DATA SCIENCE WITH PYTHON

NumPy Practice Set

100 Questions | With Solutions & Outputs

50 Medium Level + 50 Advanced Level

Arrays | Indexing | Broadcasting | Linear Algebra | Statistics | Mini Projects

Copyright SKILLARGER. For learning use by SKILLARGER students.

How to Use This Practice Set

This practice set is designed for the NumPy module of the **SKILLARGER DATA SCIENCE WITH PYTHON** course. Every question is followed by a solution and its **actual output** (all code has been executed and verified).

Section	Questions	Level	Topics
A	Q1 - Q50	Medium	Creation, Indexing, Masking, Aggregation, Broadcasting, Sorting, Linear Algebra, Random, NaN
B	Q51 - Q100	Advanced	Distance/Similarity, ML from scratch, PCA/SVD, einsum, Stride tricks, Simulation, Mini Projects

Study Tips

- Try each question on your own first, then check the solution. Spend at least 10 minutes before peeking.
- Run every code in a Jupyter Notebook / Google Colab and compare your output with the one shown.
- Try to write each solution in a second way (loop-free / vectorized).
- Random questions use a fixed seed, so your output should match exactly.

Setup (for all solutions)

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
```

Note: Some solutions import `sliding_window_view` from `numpy.lib.stride_tricks` separately. NumPy 1.20+ is required.

SECTION A - MEDIUM LEVEL

Q1 to Q50 | Core NumPy skills every data scientist needs

ARRAY CREATION

Q1. Create an array of all even numbers from 1 to 20 (using np.arange) and an array of 5 equally spaced numbers between 0 and 1 (using np.linspace).

SOLUTION

```
a = np.arange(2, 21, 2)
b = np.linspace(0, 1, 5)
print(a)
print(b)
```

OUTPUT

```
[ 2  4  6  8 10 12 14 16 18 20]
[0.   0.25 0.5  0.75 1.  ]
```

ARRAY CREATION

Q2. Create a 3x3 identity matrix, a 2x4 array of ones (dtype=int), and a 3x3 array filled with the value 7.

SOLUTION

```
print(np.eye(3))
print(np.ones((2, 4), dtype=int))
print(np.full((3, 3), 7))
```

OUTPUT

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[[1 1 1 1]
 [1 1 1 1]]
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

RESHAPE & ATTRIBUTES

Q3. Create an array from 1 to 12, reshape it to 3x4, and print its shape, ndim, size and dtype.

SOLUTION

```
a = np.arange(1, 13).reshape(3, 4)
print(a)
print(a.shape, a.ndim, a.size, a.dtype)
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
(3, 4) 2 12 int64
```

ARRAY CREATION

Q4. Create a 5x5 array with 1s on the border and 0s inside.

SOLUTION

```
a = np.ones((5, 5), dtype=int)
a[1:-1, 1:-1] = 0
print(a)
```

OUTPUT

```
[[1 1 1 1 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 1 1 1 1]]
```

ARRAY CREATION

Q5. Create an 8x8 checkerboard pattern (alternating 0 and 1) without using loops.

SOLUTION

```
a = np.zeros((8, 8), dtype=int)
a[1::2, ::2] = 1
a[::2, 1::2] = 1
print(a)
```

OUTPUT

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

INDEXING & SLICING

Q6. Reverse a 1D array. Then reverse the order of the rows of a 2D array, and also the order of its columns.

SOLUTION

```
a = np.arange(1, 7)
m = np.arange(1, 10).reshape(3, 3)
print(a[::-1])
print(m[::-1])
print(m[:, ::-1])
```

OUTPUT

```
[6 5 4 3 2 1]
[[7 8 9]
 [4 5 6]
 [1 2 3]]
[[3 2 1]
 [6 5 4]
 [9 8 7]]
```

INDEXING & SLICING

Q7. From arr = np.arange(1, 26).reshape(5, 5), extract the inner 3x3 block (rows 2 to 4 and columns 2 to 4).

SOLUTION

```
arr = np.arange(1, 26).reshape(5, 5)
print(arr[1:4, 1:4])
```

OUTPUT

```
[[ 7  8  9]
 [12 13 14]
 [17 18 19]]
```

INDEXING & SLICING

Q8. Extract the main diagonal and the anti-diagonal (opposite diagonal) of a 4x4 matrix.

SOLUTION

```
m = np.arange(16).reshape(4, 4)
print(np.diag(m))
print(np.diag(np.fliplr(m)))
```

OUTPUT

```
[ 0  5 10 15]
[ 3  6  9 12]
```

BOOLEAN MASKING

Q9. Replace all odd numbers in an array with -1 without modifying the original array.

SOLUTION

```
arr = np.arange(1, 11)
out = arr.copy()
out[out % 2 == 1] = -1
print(arr)
print(out)
```

OUTPUT

```
[ 1  2  3  4  5  6  7  8  9 10]
[-1  2 -1  4 -1  6 -1  8 -1 10]
```

BOOLEAN MASKING

Q10. Count how many elements of an array are greater than its mean, and print those elements.

SOLUTION

```
a = np.array([12, 45, 7, 33, 90, 21, 56, 3])
mask = a > a.mean()
print(a.mean(), mask.sum())
print(a[mask])
```

OUTPUT

```
33.375 3
[45 90 56]
```

BOOLEAN MASKING

Q11. marks = [35, 88, 42, 91, 67, 23, 78, 55]. Find the marks of students who scored more than 40 and less than 80.

SOLUTION

```
marks = np.array([35, 88, 42, 91, 67, 23, 78, 55])
print(marks[(marks > 40) & (marks < 80)])
```

OUTPUT

```
[42 67 78 55]
```

NP.WHERE

Q12. Replace all negative numbers in an array with 0 and keep the positive numbers unchanged (use np.where).

SOLUTION

```
a = np.array([-5, 3, -1, 8, 0, -9, 4])
print(np.where(a < 0, 0, a))
```

OUTPUT

```
[0 3 0 8 0 0 4]
```

NP.WHERE

Q13. Find the indices of the non-zero elements of an array.

SOLUTION

```
a = np.array([0, 3, 0, 5, 7, 0, 1])
print(np.nonzero(a)[0])
print(np.flatnonzero(a))
```

OUTPUT

```
[1 3 4 6]
[1 3 4 6]
```

AGGREGATION (AXIS)

Q14. Find the row-wise sum, column-wise sum and total sum of a 3x4 array.

SOLUTION

```
a = np.arange(1, 13).reshape(3, 4)
print(a.sum(axis=1))
print(a.sum(axis=0))
print(a.sum())
```

OUTPUT

```
[10 26 42]
[15 18 21 24]
78
```

STATISTICS

Q15. Find the mean, median, standard deviation, variance and range (max - min) of data = [12, 15, 11, 20, 18, 25, 30].

SOLUTION

```
d = np.array([12, 15, 11, 20, 18, 25, 30])
print(d.mean(), np.median(d))
print(d.std(), d.var())
print(np.ptp(d))
```

OUTPUT

```
18.714285714285715 18.0
6.40790328343023 41.06122448979592
19
```

NORMALIZATION

Q16. Convert an array to the 0-1 range using Min-Max normalization.

SOLUTION

```
a = np.array([10, 20, 30, 40, 50])
norm = (a - a.min()) / (a.max() - a.min())
print(norm)
```

OUTPUT

```
[0.   0.25 0.5  0.75 1.  ]
```

NORMALIZATION

Q17. Standardize a random array using Z-score and verify that the mean is ~0 and the standard deviation is 1.

SOLUTION

```
np.random.seed(0)
a = np.random.randint(10, 100, 8)
z = (a - a.mean()) / a.std()
print(a)
print(z)
print(round(z.mean(), 6), z.std())
```

OUTPUT

```
[54 57 74 77 77 19 93 31]
[-0.2656 -0.1381  0.5843  0.7118  0.7118 -1.753   1.3918 -1.2431]
-0.0 1.0
```

BROADCASTING

Q18. Add a column vector (3x1) and a row vector (1x4). What will be the shape of the result?

SOLUTION

```
col = np.array([[10], [20], [30]])
row = np.array([1, 2, 3, 4])
res = col + row
print(res)
print(res.shape)
```

OUTPUT

```
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
(3, 4)
```

BROADCASTING

Q19. Subtract each column's mean from that column of a 2D array (mean centering).

SOLUTION

```
a = np.array([[1., 2., 3.], [4., 5., 6.], [7., 8., 12.]])
centered = a - a.mean(axis=0)
print(centered)
print(centered.mean(axis=0))
```

OUTPUT

```
[[-3. -3. -4.]
 [ 0.  0. -1.]
 [ 3.  3.  5.]]
[0. 0. 0.]
```

STACKING

Q20. Stack two 2x3 arrays vertically (np.vstack) and horizontally (np.hstack).

SOLUTION

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([[7, 8, 9], [10, 11, 12]])
print(np.vstack((a, b)))
print(np.hstack((a, b)))
```

OUTPUT

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
[[ 1  2  3  7  8  9]
 [ 4  5  6 10 11 12]]
```

CONCATENATE & SPLIT

Q21. Join arrays with np.concatenate along axis=0 and axis=1, then split a 1D array into 3 equal parts using np.split.

SOLUTION

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(np.concatenate((a, b), axis=0))
print(np.concatenate((a, b), axis=1))
print(np.split(np.arange(9), 3))
```

OUTPUT

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[1 2 5 6]
 [3 4 7 8]]
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

COPY VS VIEW

Q22. What is the difference between flatten() and ravel()? Prove it with code.

SOLUTION

```
a = np.array([[1, 2], [3, 4]])
f = a.flatten()   # always a copy
r = a.ravel()     # view when possible
f[0] = 99
r[1] = 77
print(a)
print(f, r)
```

OUTPUT

```
[[ 1 77]
 [ 3  4]]
[99  2  3  4] [ 1 77  3  4]
```

COPY VS VIEW

Q23. How does an array created by slicing affect the original array? What changes when you use .copy()?

SOLUTION

```
a = np.arange(6)
view = a[1:4]
view[0] = 100
print(a)
safe = a[1:4].copy()
safe[0] = -1
print(a)
```

OUTPUT

```
[  0 100   2   3   4   5]
[  0 100   2   3   4   5]
```

SORTING

Q24. Sort an array, get the sorting indices with argsort, and find the indices of the top-3 largest values.

SOLUTION

```
a = np.array([40, 10, 90, 30, 70, 20])
print(np.sort(a))
print(np.argsort(a))
top3 = np.argsort(a)[-3:][::-1]
print(top3, a[top3])
```

OUTPUT

```
[10 20 30 40 70 90]
[1 5 3 0 4 2]
[2 4 0] [90 70 40]
```

SORTING

Q25. Sort a 2D array based on its 2nd column (index 1).

SOLUTION

```
a = np.array([[3, 9, 1], [1, 4, 8], [2, 7, 5]])
print(a[a[:, 1].argsort()])
```

OUTPUT

```
[[1 4 8]
 [2 7 5]
 [3 9 1]]
```

UNIQUE & COUNTS

Q26. Find the unique values of an array and their frequencies (how many times each occurs).

SOLUTION

```
a = np.array([3, 1, 2, 3, 3, 1, 4, 2, 3])
vals, counts = np.unique(a, return_counts=True)
print(vals)
print(counts)
```

OUTPUT

```
[1 2 3 4]
[2 2 4 1]
```

SET OPERATIONS

Q27. Find the common elements of two arrays, and the elements that are in a but not in b.

SOLUTION

```
a = np.array([1, 2, 3, 4, 5, 6])
b = np.array([4, 5, 6, 7, 8])
print(np.intersect1d(a, b))
print(np.setdiff1d(a, b))
```

OUTPUT

```
[4 5 6]
[1 2 3]
```

CUMULATIVE OPS

Q28. Find the cumulative sum, cumulative product and consecutive differences (np.diff) of an array.

SOLUTION

```
a = np.array([1, 2, 3, 4, 5])
print(np.cumsum(a))
print(np.cumprod(a))
print(np.diff(np.array([5, 9, 4, 12])))
```

OUTPUT

```
[ 1  3  6 10 15]
[  1   2   6  24 120]
[ 4 -5  8]
```

MATRIX MULTIPLICATION

Q29. Find the dot product of a 2x3 and a 3x2 matrix. Show the difference between the @ operator and the * operator.

SOLUTION

```
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[7, 8], [9, 10], [11, 12]])
print(A @ B)
print(np.dot(A, B))
x = np.array([1, 2, 3])
print(x * x, x @ x)
```

OUTPUT

```
[[ 58  64]
 [139 154]]
[[ 58  64]
 [139 154]]
[1 4 9] 14
```

LINEAR ALGEBRA

Q30. Find the transpose, trace and determinant of a 3x3 matrix.

SOLUTION

```
A = np.array([[2, 1, 3], [0, 4, 1], [5, 2, 6]])
print(A.T)
print(np.trace(A))
print(round(np.linalg.det(A), 4))
```

OUTPUT

```
[[2 0 5]
 [1 4 2]
 [3 1 6]]
12
-11.0
```

LINEAR ALGEBRA

Q31. Find the inverse of a matrix and verify that A @ A_inv equals the identity matrix.

SOLUTION

```
A = np.array([[4., 7.], [2., 6.]])
Ainv = np.linalg.inv(A)
print(Ainv)
print(np.allclose(A @ Ainv, np.eye(2)))
```

OUTPUT

```
[[ 0.6 -0.7]
 [-0.2  0.4]]
True
```

LINEAR ALGEBRA

Q32. Solve the equations 2x + y = 5 and x + 3y = 10 using np.linalg.solve.

SOLUTION

```
A = np.array([[2., 1.], [1., 3.]])
b = np.array([5., 10.])
print(np.linalg.solve(A, b))
```

OUTPUT

```
[1. 3.]
```

RANDOM MODULE

Q33. After setting a seed, generate a 3x3 random float array, 5 random integers between 1 and 100, and 4 standard normal numbers.

SOLUTION

```
np.random.seed(42)
print(np.random.rand(3, 3))
print(np.random.randint(1, 101, 5))
print(np.random.randn(4))
```

OUTPUT

```
[[0.3745 0.9507 0.732 ]
 [0.5987 0.156  0.156 ]
 [0.0581 0.8662 0.6011]]
[24  3 22 53  2]
[0.2331 0.118  1.4624 1.5387]
```

RANDOM MODULE

Q34. Shuffle an array (permutation) and choose 3 random elements without replacement.

SOLUTION

```
np.random.seed(1)
a = np.arange(10)
print(np.random.permutation(a))
print(np.random.choice(a, 3, replace=False))
```

OUTPUT

```
[2 9 6 4 0 3 1 7 8 5]
[9 5 3]
```

MISSING VALUES (NAN)

Q35. Count the NaN values in an array and replace them with the mean of the remaining values (np.nanmean).

SOLUTION

```
a = np.array([4., np.nan, 8., np.nan, 6., 10.])
print(np.isnan(a).sum())
a[np.isnan(a)] = np.nanmean(a)
print(a)
```

OUTPUT

```
2
[ 4.  7.  8.  7.  6. 10.]
```

NP.CLIP

Q36. Clip array values between 10 and 50 (values below 10 become 10, values above 50 become 50).

SOLUTION

```
a = np.array([5, 12, 48, 77, 30, 3, 55])
print(np.clip(a, 10, 50))
```

OUTPUT

```
[10 12 48 50 30 10 50]
```

ROUNDING

Q37. Apply np.round (1 decimal place), np.floor and np.ceil to a float array.

SOLUTION

```
a = np.array([1.26, 2.75, -3.41, 4.5])
print(np.round(a, 1))
print(np.floor(a))
print(np.ceil(a))
```

OUTPUT

```
[ 1.3  2.8 -3.4  4.5]
[ 1.  2. -4.  4.]
[ 2.  3. -3.  5.]
```

ARGMAX / ARGMIN

Q38. For a 2D array, find which column holds the maximum in each row (argmax, axis=1) and the position of the minimum of the whole array.

SOLUTION

```
a = np.array([[3, 9, 2], [8, 1, 7], [4, 6, 10]])
print(np.argmax(a, axis=1))
idx = np.unravel_index(np.argmin(a), a.shape)
print(idx, a[idx])
```

OUTPUT

```
[1 0 2]
(np.int64(1), np.int64(1)) 1
```

ANY / ALL

Q39. Check whether all elements of each row are positive (np.all) and whether any column contains a zero (np.any).

SOLUTION

```
a = np.array([[1, 2, 3], [4, 0, 6], [-1, 8, 9]])
print(np.all(a > 0, axis=1))
print(np.any(a == 0, axis=0))
```

OUTPUT

```
[ True False False]
[False  True False]
```

VECTORIZATION

Q40. Convert the Celsius temperatures [0, 20, 37, 100] to Fahrenheit without loops.

SOLUTION

```
c = np.array([0, 20, 37, 100])
f = c * 9 / 5 + 32
print(f)
```

OUTPUT

```
[ 32.   68.   98.6 212. ]
```

MESHGRID

Q41. Build a meshgrid from x = [0,1,2] and y = [0,1,2,3] and compute x**2 + y**2 at every point.

SOLUTION

```
x = np.arange(3)
y = np.arange(4)
X, Y = np.meshgrid(x, y)
print(X.shape)
print(X**2 + Y**2)
```

OUTPUT

```
(4, 3)
[[ 0  1  4]
 [ 1  2  5]
 [ 4  5  8]
 [ 9 10 13]]
```

TILE & REPEAT

Q42. Repeat [1, 2, 3] twice using np.tile, and repeat each element twice using np.repeat.

SOLUTION

```
a = np.array([1, 2, 3])
print(np.tile(a, 2))
print(np.repeat(a, 2))
```

OUTPUT

```
[1 2 3 1 2 3]
[1 1 2 2 3 3]
```

INSERT / DELETE

Q43. Insert 99 at index 2, delete the value at index 0, and append 50 at the end of an array.

SOLUTION

```
a = np.array([10, 20, 30, 40])
a = np.insert(a, 2, 99)
a = np.delete(a, 0)
a = np.append(a, 50)
print(a)
```

OUTPUT

```
[20 99 30 40 50]
```

SEARCHING

Q44. Find the second largest value of an array.

SOLUTION

```
a = np.array([15, 80, 42, 99, 67, 99, 23])
u = np.unique(a)
print(u[-2])
```

OUTPUT

```
80
```

DTYPE & MEMORY

Q45. Convert a float array to int using astype, and compare the nbytes of int64 and int16 arrays.

SOLUTION

```
a = np.array([1.9, 2.5, -3.7])
print(a.astype(int))
x = np.arange(1000)
print(x.astype(np.int64).nbytes, x.astype(np.int16).nbytes)
```

OUTPUT

```
[ 1  2 -3]
8000 2000
```

KEEPDIMS

Q46. Divide each row by its row-sum to get row-wise percentages (use keepdims).

SOLUTION

```
a = np.array([[10, 20, 70], [5, 5, 10]])
pct = a / a.sum(axis=1, keepdims=True) * 100
print(pct)
print(pct.sum(axis=1))
```

OUTPUT

```
[[10. 20. 70.]
 [25. 25. 50.]]
[100. 100.]
```

SEARCHING

Q47. Find the value in an array that is closest to the target 33 and report its index.

SOLUTION

```
a = np.array([5, 18, 31, 40, 27, 90])
t = 33
i = np.abs(a - t).argmin()
print(i, a[i])
```

OUTPUT

```
2 31
```

EYE / TRIU

Q48. Create a 4x4 matrix with ones on the first diagonal above the main diagonal (np.eye with k=1), and extract the upper triangular part of a matrix (np.triu).

SOLUTION

```
print(np.eye(4, k=1, dtype=int))
m = np.arange(1, 10).reshape(3, 3)
print(np.triu(m))
```

OUTPUT

```
[[0 1 0 0]
 [0 0 1 0]
 [0 0 0 1]
 [0 0 0 0]]
[[1 2 3]
 [0 5 6]
 [0 0 9]]
```

BINCOUNT

Q49. Use np.bincount to find the frequency of each integer in an array.

SOLUTION

```
a = np.array([0, 1, 1, 3, 2, 1, 3, 3, 3])
print(np.bincount(a))
```

OUTPUT

```
[1 3 1 4]
```

MOVING AVERAGE

Q50. Compute a simple moving average with window size 3 using np.convolve.

SOLUTION

```
a = np.array([1., 2., 3., 4., 5., 6., 7.])
ma = np.convolve(a, np.ones(3) / 3, mode='valid')
print(ma)
```

OUTPUT

```
[2. 3. 4. 5. 6.]
```

SECTION B - ADVANCED LEVEL

Q51 to Q100 | Vectorization, ML from scratch, real-world problems

BROADCASTING / DISTANCE

Q51. Compute the pairwise Euclidean distance matrix between 5 two-dimensional points without any loops (use broadcasting).

SOLUTION

```
np.random.seed(0)
P = np.random.randint(0, 10, (5, 2))
diff = P[:, None, :] - P[None, :, :]
D = np.sqrt((diff ** 2).sum(axis=-1))
print(P)
print(D.round(2))
```

OUTPUT

```
[[5 0]
 [3 3]
 [7 9]
 [3 5]
 [2 4]]
[[0.   3.61 9.22 5.39 5.  ]
 [3.61 0.   7.21 2.   1.41]
 [9.22 7.21 0.   5.66 7.07]
 [5.39 2.   5.66 0.   1.41]
 [5.   1.41 7.07 1.41 0.  ]]
```

COSINE SIMILARITY

Q52. Compute the cosine similarity matrix between the rows of a matrix.

SOLUTION

```
X = np.array([[1., 2., 3.], [2., 4., 6.], [3., 0., 1.]])
Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
S = Xn @ Xn.T
print(S.round(3))
```

OUTPUT

```
[[1.    1.    0.507]
 [1.    1.    0.507]
 [0.507 0.507 1.   ]]
```

SOFTMAX

Q53. Write a numerically stable row-wise softmax function (subtract the max) and check that every row sums to 1.

SOLUTION

```
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

z = np.array([[1000., 1001., 1002.], [1., 2., 3.]])
s = softmax(z)
print(s.round(4))
print(s.sum(axis=1))
```

OUTPUT

```
[[0.09   0.2447 0.6652]
 [0.09   0.2447 0.6652]]
[1. 1.]
```

ONE-HOT ENCODING

Q54. One-hot encode an array of class labels (using np.eye indexing).

SOLUTION

```
y = np.array([0, 2, 1, 2, 0])
n_classes = y.max() + 1
print(np.eye(n_classes, dtype=int)[y])
```

OUTPUT

```
[[1 0 0]
 [0 0 1]
 [0 1 0]
 [0 0 1]
 [1 0 0]]
```

FEATURE SCALING

Q55. Write fit and transform functions for a column-wise MinMax scaler (transform the test data using the train min/max).

SOLUTION

```
def fit(X):
    return X.min(axis=0), X.max(axis=0)

def transform(X, mn, mx):
    return (X - mn) / (mx - mn)

train = np.array([[10., 200.], [20., 400.], [30., 600.]])
test = np.array([[15., 500.]])
mn, mx = fit(train)
print(transform(train, mn, mx))
print(transform(test, mn, mx))
```

OUTPUT

```
[[0.  0. ]
 [0.5 0.5]
 [1.  1. ]]
[[0.25 0.75]]
```

LINEAR REGRESSION

Q56. Fit a linear regression using the normal equation theta = (X^T X)^-1 X^T y. Data: y = 3 + 2x + noise.

SOLUTION

```
rng = np.random.default_rng(0)
x = rng.uniform(0, 10, 50)
y = 3 + 2 * x + rng.normal(0, 0.5, 50)
X = np.column_stack([np.ones_like(x), x])
theta = np.linalg.inv(X.T @ X) @ X.T @ y
print(theta.round(3))
print(np.linalg.lstsq(X, y, rcond=None)[0].round(3))
```

OUTPUT

```
[2.879 2.026]
[2.879 2.026]
```

GRADIENT DESCENT

Q57. Learn linear regression weights with vectorized Gradient Descent (learning rate 0.01, 2000 iterations).

SOLUTION

```
rng = np.random.default_rng(1)
x = rng.uniform(0, 5, 100)
y = 4 + 3 * x + rng.normal(0, 0.3, 100)
X = np.column_stack([np.ones(100), x])
w = np.zeros(2)
for _ in range(2000):
    grad = (2 / 100) * X.T @ (X @ w - y)
    w -= 0.01 * grad
print(w.round(3))
```

OUTPUT

```
[3.979 3.001]
```

PCA FROM SCRATCH

Q58. Perform PCA using the covariance matrix and eigen decomposition. Report the explained variance ratio of each component.

SOLUTION

```
rng = np.random.default_rng(2)
A = rng.normal(0, 1, (100, 3))
X = A @ np.array([[3, 0, 0], [1, 1, 0], [0.5, 0.2, 0.1]])
Xc = X - X.mean(axis=0)
cov = np.cov(Xc, rowvar=False)
vals, vecs = np.linalg.eigh(cov)
order = np.argsort(vals)[::-1]
vals, vecs = vals[order], vecs[:, order]
print((vals / vals.sum()).round(4))
Z = Xc @ vecs[:, :2]
print(Z.shape)
```

OUTPUT

```
[0.9215 0.0778 0.0007]
(100, 2)
```

SVD

Q59. Build a rank-1 approximation of a matrix using SVD and compute the reconstruction error (Frobenius norm).

SOLUTION

```
A = np.array([[3., 1., 1.], [-1., 3., 1.], [2., 2., 2.]])
U, s, Vt = np.linalg.svd(A)
A1 = s[0] * np.outer(U[:, 0], Vt[0])
print(s.round(3))
print(np.linalg.norm(A - A1).round(4))
```

OUTPUT

```
[4.859 3.181 0.518]
3.2231
```

CORRELATION

Q60. Compute a correlation matrix with np.corrcoef and verify the correlation of two variables manually using the formula.

SOLUTION

```
x = np.array([1., 2., 3., 4., 5.])
y = np.array([2., 4., 5., 4., 5.])
print(np.corrcoef(x, y).round(4))
manual = ((x - x.mean()) * (y - y.mean())).sum() / np.sqrt(((x - x.mean())**2).sum() * ((y -
     y.mean())**2).sum())
print(round(manual, 4))
```

OUTPUT

```
[[1.     0.7746]
 [0.7746 1.    ]]
0.7746
```

SLIDING_WINDOW_VIEW

Q61. Compute a rolling mean with window=4 on a 1D array using sliding_window_view.

SOLUTION

```
from numpy.lib.stride_tricks import sliding_window_view as swv
a = np.array([2., 4., 6., 8., 10., 12., 14.])
w = swv(a, 4)
print(w)
print(w.mean(axis=1))
```

OUTPUT

```
[[ 2.  4.  6.  8.]
 [ 4.  6.  8. 10.]
 [ 6.  8. 10. 12.]
 [ 8. 10. 12. 14.]]
[ 5.  7.  9. 11.]
```

ROLLING MAX

Q62. Compute a rolling maximum and a rolling range (max - min) with window=3 using sliding_window_view.

SOLUTION

```
from numpy.lib.stride_tricks import sliding_window_view as swv
a = np.array([5, 1, 8, 3, 9, 2, 7])
w = swv(a, 3)
print(w.max(axis=1))
print(w.max(axis=1) - w.min(axis=1))
```

OUTPUT

```
[8 8 9 9 9]
[7 7 6 7 7]
```

EINSUM BASICS

Q63. Use np.einsum to perform matrix multiplication, trace and transpose.

SOLUTION

```
A = np.arange(6).reshape(2, 3)
B = np.arange(6).reshape(3, 2)
print(np.einsum('ij,jk->ik', A, B))
M = np.arange(9).reshape(3, 3)
print(np.einsum('ii->', M))
print(np.einsum('ij->ji', A))
```

OUTPUT

```
[[10 13]
 [28 40]]
12
[[0 3]
 [1 4]
 [2 5]]
```

EINSUM ADVANCED

Q64. Use einsum to compute the row-wise dot product and the outer product.

SOLUTION

```
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[1, 0, 1], [2, 1, 0]])
print(np.einsum('ij,ij->i', A, B))
u = np.array([1, 2, 3])
v = np.array([4, 5])
print(np.einsum('i,j->ij', u, v))
```

OUTPUT

```
[ 4 13]
[[ 4  5]
 [ 8 10]
 [12 15]]
```

BATCH MATMUL

Q65. Perform batch matrix multiplication of 3D arrays with shapes (2,3,4) and (2,4,2). State the output shape.

SOLUTION

```
np.random.seed(3)
A = np.random.randint(0, 5, (2, 3, 4))
B = np.random.randint(0, 5, (2, 4, 2))
C = np.matmul(A, B)
print(C.shape)
print(C[0])
```

OUTPUT

```
(2, 3, 2)
[[17  4]
 [ 9  0]
 [11  7]]
```

FANCY INDEXING

Q66. Use fancy indexing to select specific (row, column) pairs of a matrix and modify their values.

SOLUTION

```
m = np.arange(16).reshape(4, 4)
rows = np.array([0, 1, 3])
cols = np.array([2, 0, 3])
print(m[rows, cols])
m[rows, cols] = -1
print(m)
```

OUTPUT

```
[ 2  4 15]
[[ 0  1 -1  3]
 [-1  5  6  7]
 [ 8  9 10 11]
 [12 13 14 -1]]
```

NP.SELECT

Q67. Assign grades from marks: >=90 'A', >=75 'B', >=50 'C', otherwise 'F' (use np.select).

SOLUTION

```
m = np.array([95, 82, 60, 45, 77, 91])
conds = [m >= 90, m >= 75, m >= 50]
print(np.select(conds, ['A', 'B', 'C'], default='F'))
```

OUTPUT

```
['A' 'B' 'C' 'F' 'B' 'A']
```

DIGITIZE & HISTOGRAM

Q68. Place ages into the bins [0, 18, 35, 60, 100] with np.digitize and count them per bin with np.histogram.

SOLUTION

```
ages = np.array([5, 17, 18, 25, 34, 35, 42, 59, 61, 80])
bins = np.array([0, 18, 35, 60, 100])
print(np.digitize(ages, bins))
counts, edges = np.histogram(ages, bins=bins)
print(counts, edges)
```

OUTPUT

```
[1 1 2 2 2 3 3 3 4 4]
[2 3 3 2] [  0  18  35  60 100]
```

IQR OUTLIERS

Q69. Detect outliers using the IQR method (Q1 - 1.5*IQR, Q3 + 1.5*IQR).

SOLUTION

```
d = np.array([10, 12, 11, 13, 12, 95, 11, 14, 10, -40])
q1, q3 = np.percentile(d, [25, 75])
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(q1, q3, lo, hi)
print(d[(d < lo) | (d > hi)])
```

OUTPUT

```
10.25 12.75 6.5 16.5
[ 95 -40]
```

Z-SCORE OUTLIERS

Q70. In a 2D dataset, remove every row that has any value with |z-score| > 2.

SOLUTION

```
X = np.array([[10, 100], [12, 110], [11, 105], [13, 95], [11, 108], [10, 106],
              [12, 99], [11, 102], [13, 101], [12, 104], [60, 400]], dtype=float)
z = (X - X.mean(axis=0)) / X.std(axis=0)
keep = (np.abs(z) <= 2).all(axis=1)
print(keep)
print(X[keep].shape)
```

OUTPUT

```
[ True  True  True  True  True  True  True  True  True  True False]
(10, 2)
```

MAX DRAWDOWN

Q71. Find the maximum drawdown of a stock price series (use np.maximum.accumulate).

SOLUTION

```
p = np.array([100, 110, 105, 120, 90, 95, 130, 100], dtype=float)
peak = np.maximum.accumulate(p)
dd = (p - peak) / peak
print(dd.round(3))
print(dd.min().round(4))
```

OUTPUT

```
[ 0.     0.    -0.045  0.    -0.25  -0.208  0.    -0.231]
-0.25
```

RETURNS & VOLATILITY

Q72. Compute daily returns from a price series and the annualized volatility (std * sqrt(252)).

SOLUTION

```
p = np.array([100., 102., 101., 105., 107., 104., 108.])
r = p[1:] / p[:-1] - 1
print(r.round(4))
print((r.std(ddof=1) * np.sqrt(252)).round(4))
```

OUTPUT

```
[ 0.02   -0.0098  0.0396  0.019  -0.028   0.0385]
0.4286
```

STRUCTURED ARRAY

Q73. Create a structured array (name, age, marks) and sort it in descending order of marks.

SOLUTION

```
dt = [('name', 'U8'), ('age', 'i4'), ('marks', 'f4')]
s = np.array([('Amit', 21, 78.5), ('Riya', 20, 91.0), ('Neha', 22, 66.0)], dtype=dt)
print(np.sort(s, order='marks')[::-1])
print(s['marks'].mean())
```

OUTPUT

```
[('Riya', 20, 91. ) ('Amit', 21, 78.5) ('Neha', 22, 66. )]
78.5
```

MASKED ARRAYS

Q74. Use np.ma to compute the mean while ignoring the invalid value -999.

SOLUTION

```
a = np.array([10, 20, -999, 30, -999, 40])
m = np.ma.masked_equal(a, -999)
print(m)
print(m.mean(), a.mean())
```

OUTPUT

```
[10 20 -- 30 -- 40]
25.0 -316.3333333333333
```

NAN IMPUTATION

Q75. Fill the NaN values in every column of a 2D array with that column's mean (vectorized).

SOLUTION

```
X = np.array([[1., np.nan, 3.], [4., 5., np.nan], [7., 8., 9.]])
col_mean = np.nanmean(X, axis=0)
X_filled = np.where(np.isnan(X), col_mean, X)
print(X_filled)
```

OUTPUT

```
[[1.  6.5 3. ]
 [4.  5.  6. ]
 [7.  8.  9. ]]
```

ARGPARTITION

Q76. Find the top-3 largest values without fully sorting the array (np.argpartition).

SOLUTION

```
a = np.array([7, 1, 9, 3, 15, 2, 11, 6])
idx = np.argpartition(a, -3)[-3:]
print(np.sort(a[idx])[::-1])
```

OUTPUT

```
[15 11  9]
```

LEXSORT

Q77. Multi-key sorting: sort by city (A-Z) first, then by salary (high to low) using np.lexsort.

SOLUTION

```
city = np.array(['Pune', 'Delhi', 'Pune', 'Delhi', 'Pune'])
salary = np.array([50, 70, 90, 60, 20])
order = np.lexsort((-salary, city))
print(city[order])
print(salary[order])
```

OUTPUT

```
['Delhi' 'Delhi' 'Pune' 'Pune' 'Pune']
[70 60 90 50 20]
```

RANKING

Q78. Find the rank of each element (1 = smallest) by applying argsort twice.

SOLUTION

```
a = np.array([40, 10, 30, 20])
rank = a.argsort().argsort() + 1
print(rank)
```

OUTPUT

```
[4 1 3 2]
```

RUN-LENGTH ENCODING

Q79. Compute the Run-Length Encoding of an array (each value and how many times it repeats consecutively).

SOLUTION

```
a = np.array([1, 1, 2, 2, 2, 3, 1, 1])
change = np.flatnonzero(np.diff(a)) + 1
starts = np.r_[0, change]
lengths = np.diff(np.r_[starts, len(a)])
print(a[starts])
print(lengths)
```

OUTPUT

```
[1 2 3 1]
[2 3 1 2]
```

LONGEST RUN OF 1S

Q80. Find the length of the longest run of consecutive 1s in a binary array.

SOLUTION

```
a = np.array([1, 1, 0, 1, 1, 1, 0, 1])
padded = np.r_[0, a, 0]
d = np.diff(padded)
starts = np.flatnonzero(d == 1)
ends = np.flatnonzero(d == -1)
print((ends - starts).max())
```

OUTPUT

```
3
```

GAME OF LIFE

Q81. Compute one generation of Conway's Game of Life in a vectorized way (using np.roll).

SOLUTION

```
g = np.zeros((5, 5), dtype=int)
g[2, 1:4] = 1   # blinker
n = sum(np.roll(np.roll(g, i, 0), j, 1)
        for i in (-1, 0, 1) for j in (-1, 0, 1) if (i, j) != (0, 0))
new = ((g == 1) & ((n == 2) | (n == 3))) | ((g == 0) & (n == 3))
print(new.astype(int))
```

OUTPUT

```
[[0 0 0 0 0]
 [0 0 1 0 0]
 [0 0 1 0 0]
 [0 0 1 0 0]
 [0 0 0 0 0]]
```

ROLL / ROT90 / FLIP

Q82. Circularly shift an array (np.roll), rotate it by 90 degrees (np.rot90) and flip it (np.flipud).

SOLUTION

```
a = np.arange(1, 10).reshape(3, 3)
print(np.roll(np.arange(5), 2))
print(np.rot90(a))
print(np.flipud(a))
```

OUTPUT

```
[3 4 0 1 2]
[[3 6 9]
 [2 5 8]
 [1 4 7]]
[[7 8 9]
 [4 5 6]
 [1 2 3]]
```

2D CONVOLUTION

Q83. Compute the 'valid' convolution of a 5x5 image with a 3x3 kernel using sliding_window_view and einsum.

SOLUTION

```
from numpy.lib.stride_tricks import sliding_window_view as swv
img = np.arange(25).reshape(5, 5)
k = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]])
win = swv(img, (3, 3))
out = np.einsum('ijkl,kl->ij', win, k)
print(out)
```

OUTPUT

```
[[0 0 0]
 [0 0 0]
 [0 0 0]]
```

MONTE CARLO

Q84. Estimate the value of pi with a Monte Carlo simulation (1 million random points).

SOLUTION

```
rng = np.random.default_rng(7)
pts = rng.random((1_000_000, 2))
inside = (pts ** 2).sum(axis=1) <= 1
print(4 * inside.mean())
```

OUTPUT

```
3.143496
```

RANDOM WALK

Q85. Simulate a 1D random walk (1000 steps of +1/-1) and find the final position and the maximum distance from the origin.

SOLUTION

```
rng = np.random.default_rng(3)
steps = rng.choice([-1, 1], 1000)
walk = np.cumsum(steps)
print(walk[-1], np.abs(walk).max(), walk.max(), walk.min())
```

OUTPUT

```
-18 22 10 -22
```

BOOTSTRAP

Q86. Compute a 95% confidence interval for the sample mean using bootstrap (1000 resamples).

SOLUTION

```
rng = np.random.default_rng(5)
data = rng.normal(50, 10, 40)
idx = rng.integers(0, 40, (1000, 40))
means = data[idx].mean(axis=1)
print(data.mean().round(3))
print(np.percentile(means, [2.5, 97.5]).round(3))
```

OUTPUT

```
46.778
[44.323 49.496]
```

GENERATOR API

Q87. Use np.random.default_rng to generate normal numbers, integers and a choice with probabilities (p).

SOLUTION

```
rng = np.random.default_rng(42)
print(rng.normal(0, 1, 3).round(3))
print(rng.integers(1, 7, 5))
print(rng.choice(['red', 'green', 'blue'], 6, p=[0.6, 0.3, 0.1]))
```

OUTPUT

```
[ 0.305 -1.04   0.75 ]
[1 5 2 1 4]
['green' 'green' 'red' 'red' 'red' 'blue']
```

CUMSUM TRICK

Q88. Compute a moving average with window=3 in O(n) using the cumsum trick.

SOLUTION

```
a = np.array([1., 3., 5., 7., 9., 11.])
c = np.cumsum(np.insert(a, 0, 0))
w = 3
print((c[w:] - c[:-w]) / w)
```

OUTPUT

```
[3. 5. 7. 9.]
```

NORMS

Q89. Normalize every row of a matrix to unit length (L2 norm = 1).

SOLUTION

```
X = np.array([[3., 4.], [1., 1.], [0., 5.]])
n = np.linalg.norm(X, axis=1)
U = X / n[:, None]
print(n.round(3))
print(U.round(3))
print(np.linalg.norm(U, axis=1))
```

OUTPUT

```
[5.    1.414 5.   ]
[[0.6   0.8  ]
 [0.707 0.707]
 [0.    1.   ]]
[1. 1. 1.]
```

EIGENVALUES

Q90. Find the eigenvalues and eigenvectors of a symmetric matrix and verify that A v = lambda v.

SOLUTION

```
A = np.array([[4., 1.], [1., 3.]])
vals, vecs = np.linalg.eigh(A)
print(vals.round(4))
v = vecs[:, 1]
print(np.allclose(A @ v, vals[1] * v))
```

OUTPUT

```
[2.382 4.618]
True
```

MATRIX POWER

Q91. Find the 30th Fibonacci number using matrix exponentiation (np.linalg.matrix_power).

SOLUTION

```
n = 30
M = np.linalg.matrix_power(np.array([[1, 1], [1, 0]]), n)
print(M[0, 1])
```

OUTPUT

```
832040
```

POLYNOMIAL FIT

Q92. Fit a degree-2 polynomial with np.polyfit and predict the value at x = 5 (np.polyval).

SOLUTION

```
x = np.arange(0, 6, dtype=float)
y = 2 * x**2 + 3 * x + 1
coef = np.polyfit(x, y, 2)
print(coef.round(3))
print(round(np.polyval(coef, 5), 4))
```

OUTPUT

```
[2. 3. 1.]
66.0
```

NP.PIECEWISE

Q93. Vectorize the function f(x) = x**2 for x < 0, 0 for x == 0, and 2*x for x > 0 using np.piecewise.

SOLUTION

```
x = np.array([-3, -1, 0, 2, 5])
f = np.piecewise(x, [x < 0, x == 0, x > 0],
                 [lambda t: t**2, 0, lambda t: 2 * t])
print(f)
```

OUTPUT

```
[ 9  1  0  4 10]
```

BROADCASTING RULES

Q94. What is the result shape when the shapes (8,1,6,1) and (7,1,5) are broadcast? Verify with np.broadcast_shapes. Why does (3,4) with (3,) fail?

SOLUTION

```
print(np.broadcast_shapes((8, 1, 6, 1), (7, 1, 5)))
try:
    np.broadcast_shapes((3, 4), (3,))
except ValueError:
    print('ValueError: shapes (3,4) and (3,) not compatible')
print(np.broadcast_shapes((3, 4), (3, 1)))
```

OUTPUT

```
(8, 7, 6, 5)
ValueError: shapes (3,4) and (3,) not compatible
(3, 4)
```

MEMORY OPTIMIZATION

Q95. Save memory by downcasting data to a smaller dtype. Check the limits with np.iinfo first.

SOLUTION

```
a = np.arange(0, 200, dtype=np.int64)
print(np.iinfo(np.uint8).min, np.iinfo(np.uint8).max)
b = a.astype(np.uint8)
print(a.nbytes, b.nbytes)
print((a == b).all())
```

OUTPUT

```
0 255
1600 200
True
```

MULTI-CONDITION FILTER

Q96. Filter the rows of 2D data where col0 > 25 AND col2 < 60000.

SOLUTION

```
data = np.array([[22, 5, 40000], [30, 8, 55000], [41, 12, 75000],
                 [28, 3, 52000], [35, 9, 61000]])
mask = (data[:, 0] > 25) & (data[:, 2] < 60000)
print(data[mask])
```

OUTPUT

```
[[   30     8 55000]
 [   28     3 52000]]
```

TRIU_INDICES

Q97. Compute the distances between all unique pairs (np.triu_indices) and count the pairs closer than a threshold.

SOLUTION

```
pts = np.array([0., 1., 5., 6., 12.])
D = np.abs(pts[:, None] - pts[None, :])
i, j = np.triu_indices(len(pts), k=1)
d = D[i, j]
print(d)
print((d < 2).sum())
```

OUTPUT

```
[ 1.  5.  6. 12.  4.  5. 11.  1.  7.  6.]
2
```

GROUP-BY WITH BINCOUNT

Q98. Compute group-wise means without pandas: labels [0,1,0,2,1,2,0] and values [10,20,30,40,50,60,70].

SOLUTION

```
g = np.array([0, 1, 0, 2, 1, 2, 0])
v = np.array([10., 20., 30., 40., 50., 60., 70.])
sums = np.bincount(g, weights=v)
cnt = np.bincount(g)
print(sums / cnt)
```

OUTPUT

```
[36.6667 35.     50.    ]
```

MINI PROJECT: K-MEANS

Q99. Run a vectorized K-Means (k=2) for 10 iterations: assign clusters, then update the centroids.

SOLUTION

```
rng = np.random.default_rng(0)
X = np.vstack([rng.normal(0, 0.5, (20, 2)), rng.normal(5, 0.5, (20, 2))])
C = X[[0, 20]].copy()
for _ in range(10):
    d = np.linalg.norm(X[:, None, :] - C[None, :, :], axis=2)
    lab = d.argmin(axis=1)
    C = np.array([X[lab == k].mean(axis=0) for k in range(2)])
print(C.round(3))
print(np.bincount(lab))
```

OUTPUT

```
[[-0.135  0.075]
 [ 5.201  5.1  ]]
[20 20]
```

MINI PROJECT: SALES ANALYTICS

Q100. Using only NumPy, analyze a sales dataset (product, month, units, price): total revenue, best product, month-wise revenue and the top month.

SOLUTION

```
prod = np.array([0, 1, 2, 0, 1, 2, 0, 1, 2])   # 3 products
month = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2])
units = np.array([10, 5, 8, 12, 7, 6, 9, 11, 4])
price = np.array([100., 250., 80., 100., 250., 80., 100., 250., 80.])
rev = units * price
print(rev.sum())
print(np.bincount(prod, weights=rev))
print(np.bincount(month, weights=rev))
print('best product:', np.bincount(prod, weights=rev).argmax())
print('top month:', np.bincount(month, weights=rev).argmax())
```

OUTPUT

```
10290.0
[3100. 5750. 1440.]
[2890. 3430. 3970.]
best product: 1
top month: 2
```

Keep Practicing!

You have completed 100 NumPy problems. Now move on to the Pandas and Matplotlib modules.

SKILLARGER - Learn. Build. Get Hired.